The output will vary according to your preferred OS, user ID, phase of the moon, and so on. Here are some examples:

```
#Debian 13
php  54543  pault 4u  IPv6  81840              0t0  TCP [::1]:8080 (LISTEN)

#Fedora 42
php  1794   pault 4u  IPv6  26225              0t0  TCP [::1]:8080 (LISTEN)

#MacOS Sonoma
php  66720  pault 7u  IPv6  0xf5d86a0dc918232d  0t0  TCP [::1]:8080 (LISTEN)
```

In all cases we can see that `php` is listening on TCP 8080 to the local loop-back address (Debian using IPv4, the others IPv6). Only traffic originating from the local machine will get a response. If you want PHP to accept connections from any address, use this command: `php -S 0.0.0.0:8080` (TCP 8080, any IPv4 address) or `php -S [::]:8080` (TCP 8080, any IPv6 address). Be aware that this will expose port 8080 to all traffic, so be careful.

It is also possible to run a dockerized PHP binary in `cli-server` mode, but bear in mind the networking will be a little more complex. The container will need a port forward from the host, and will need to accept non-local traffic. The following docker command demonstrates how to do it:

```
$ docker run -it --rm \
  -p 8080:8080 \
  -v "$PWD":/usr/src/myapp \
  -w /usr/src/myapp \
  php:8.4-cli \
  php -S 0.0.0.0:8080
```

The `-p 8080:8080` "switch" (specific options that control command behavior) maps host TCP 8080 to container TCP 8080, and the final `php -S 0.0.0.0:8080` portion of the command is the invocation for the containerized PHP process. Make sure the port mapping with the `-p` switch aligns with the listening port inside the container.

When PHP is running in `cli-server` mode its default configuration is to use the current directory as the document root, and will use `index.php` as the default document. If you invoked `php -S` in the directory where we previously set up `php-fpm` and `nginx` containers then there's an `index.php` already available.